

# Manual Acadêmico Consolidado

Este documento reúne de forma contínua e sem interrupções os conceitos essenciais de processos de engenharia, arquitetura de sistemas orientados a objetos, validação estrutural e modelagem conceitual UML.

## 1. Requisitos, Ciclos de Vida e Restrições de Projeto

### 1.1. Restrições de Projeto

As restrições de projeto representam requisitos de sistema capturados durante a fase de engenharia de requisitos (concepção e elaboração). São limitações, imposições ou condições externas impostas que podem afetar o hardware, o software, a estrutura de dados ou os procedimentos operacionais.

Diferenciam-se dos objetivos do sistema por serem fatores impeditivos ou limitadores. Por exemplo, embora um objetivo de negócio seja a portabilidade total, a restrição de segurança ou de hardware legado pode limitar ou impedir certas implementações.

### 1.2. O Papel da Prototipagem

O protótipo é uma representação simplificada ou preliminar do sistema, criada com a finalidade de permitir a visualização, experimentação e validação de funcionalidades antes da codificação definitiva. É utilizado primariamente para esclarecer requisitos ambíguos, facilitar a comunicação entre a equipe técnica e as partes interessadas, e testar a usabilidade das interfaces. Contudo, traz o risco inerente de induzir os utilizadores a sugerir requisitos secundários ou não prioritários prematuramente.

### 1.3. Processos Iterativos e Incrementais

Em oposição aos modelos lineares puramente sequenciais (como o tradicional Modelo em Cascata), o desenvolvimento iterativo fragmenta o desenvolvimento em ciclos menores e repetidos (sprints ou iterações). Cada ciclo produz um incremento estável e testável do software. Isso reduz substancialmente riscos, acelera a entrega de valor de negócio e permite ajustes dinâmicos à medida que os requisitos evoluem.

## 2. Princípios Fundamentais do Paradigma de Orientação a Objetos (POO)

---

A programação orientada a objetos (POO) modulariza sistemas baseando-se no conceito de **Classes** e **Objetos**. Uma classe serve como o modelo abstrato (molde), definindo os atributos (características) e os métodos (comportamentos). O objeto é uma instância concreta gerada a partir desse molde.

**A) Sobrecarga (Overloading):** Refere-se à capacidade de definir, numa mesma classe, múltiplas versões de um método com o mesmo nome, desde que as suas assinaturas (quantidade ou tipo de parâmetros de entrada) sejam distintas. Exemplo clássico em linguagens tipadas:

```
// Construtor sem parâmetros (Padrão)
public Point() { ... }

// Construtor sobrecarregado recebendo coordenadas
public Point(int x, int y) { ... }
```

**B) Herança (Inheritance):** É o mecanismo que possibilita a uma subclasse (classe filha) herdar os atributos e métodos de uma superclasse (classe pai), promovendo o reuso extensivo de lógica e código já validados. A relação é estritamente unidirecional.

**C) Sobreposição (Overriding / Sobrescrita):** Ocorre quando uma subclasse redefine a implementação exata de um método herdado da superclasse, mantendo a mesma assinatura. Permite que classes filhas expressem comportamentos específicos para uma mesma operação genérica.

**D) Abstração:** Consiste na capacidade de isolar e modelar apenas os aspetos e propriedades dos objetos que são relevantes para o domínio do problema em questão, ignorando detalhes desnecessários ou complexidades internas.

**E) Troca de Mensagens:** A interação dinâmica e comunicação entre os objetos na POO ocorre através da invocação mútua de métodos, que constitui conceitualmente o envio de mensagens entre componentes autónomos.

### 3. Diagramas de Sequência na Modelagem UML

---

O Diagrama de Sequência é um diagrama comportamental da UML direcionado para evidenciar a ordem temporal em que os objetos trocam mensagens para realizar um determinado cenário ou caso de uso.

**Linha de Vida (Lifeline):** Representada por uma linha tracejada vertical que indica o tempo de existência de uma instância/objeto no decorrer do processo.

**Foco de Controle (Activation Bar):** Um retângulo estreito sobre a linha de vida que demonstra que o objeto está ativamente a processar uma instrução ou à espera de uma resposta.

**Autochamada (Self-call):** Quando um método invoca uma rotina ou operação interna da sua própria classe.

**Destruição de Objetos:** Indicada graficamente por um "X" no fim da linha de vida, marcando o instante em que a instância é desalocada da memória (destruída).

**Fragmentos Combinados (Interaction Frames):** Estruturas que definem comportamentos condicionais ou loops no fluxo de mensagens, tais como **loop** (laços de repetição), **alt** (condicionais se/senão), ou **par** (operações paralelas ou concorrentes).

## 4. Verificação, Validação e Testes de Software

### Teste de Caixa-Branca (Estrutural)

Foca-se na análise lógica e na estrutura interna do código-fonte. O analista que executa este teste necessita de possuir amplo conhecimento de programação e da implementação do produto para validar os caminhos de controle, fluxo de dados, condições lógicas, ciclos e linhas de código específicas.

Exemplos: Testes unitários, análise de cobertura de código, testes de mutação.

### Teste de Caixa-Preta (Funcional)

Trata o software como uma entidade cujas estruturas de código internas não são consideradas. Avalia-se apenas se o sistema responde de acordo com as especificações funcionais e os requisitos estipulados diante de determinadas entradas.

Exemplos: Testes de sistema, testes de aceitação de utilizador, testes de fumaça (smoke tests).

## 5. Conceito e Arquitetura de Máquinas Virtuais (MV)

---

Uma máquina virtual (MV) é um ambiente computacional completamente emulado através de software que se comporta, perante o sistema operativo nela instalado, como se fosse um hardware físico real e independente (com a sua própria CPU, memória, armazenamento e placa de rede emulados).

### Principais Vantagens

- Isolamento absoluto entre ambientes virtuais.
- Independência de hardware hospedeiro específico.
- Aumento substancial da segurança e controlo.
- Consolidação e consequente redução de custos de servidores físicos.

### Limitações e Custos

- Introdução de uma camada extra de software (sobrecarga ou *overhead*).
- Menor desempenho comparativamente a execuções diretas sobre o metal (bare-metal).
- Dependência estrita da capacidade do processador físico.

## 6. Processo Unificado (RUP) e as suas Dimensões

---

O RUP (Rational Unified Process) é uma metodologia tradicional de engenharia de software estruturada sobre um paradigma iterativo e incremental, caracterizada pelo planeamento orientado por casos de uso e centrado em arquitetura.

**1. Dimensão Dinâmica:** Representa o aspeto temporal do projeto através de ciclos divididos em quatro grandes fases sequenciais, cada qual terminando num marco (milestone) de decisão:

- **Concepção (Inception):** Definição do escopo, viabilidade e modelo de negócio do sistema.
- **Elaboração (Elaboration):** Mitigação de riscos técnicos e especificação da baseline da arquitetura.
- **Construção (Construction):** Codificação, integração de componentes e testes das funcionalidades.
- **Transição (Transition):** Implantação, homologação e disponibilização operacional para os utilizadores finais.

**2. Dimensão Estática:** Representa as disciplinas técnicas e atividades de engenharia transversais executadas em paralelo durante as fases (Modelagem de Negócio, Requisitos, Análise e Design, Implementação, Teste, Implantação e Gestão de Projeto). A execução destas disciplinas gera, de forma obrigatória, diversos **Artefactos** (especificações, esquemas de bases de dados, protótipos e diagramas).

## 7. Técnicas para Captura e Elicitação de Requisitos

---

A elicitação de requisitos visa compreender o universo do utilizador para estruturar com precisão a funcionalidade pretendida. Existem diversas técnicas consolidadas na engenharia de software:

**JAD (Joint Application Design):** Técnica baseada em sessões presenciais conjuntas e estruturadas entre analistas e utilizadores com foco em visões incrementalmente refinadas (geral, macro e detalhe).

**FAST (Facilitated Application Specification Techniques):** Reuniões de equipa assistidas por um facilitador neutro, com o propósito de harmonizar as expectativas do cliente final e as possibilidades do programador.

**PIECES:** Modelo analítico estruturado focado na identificação sistemática de fraquezas e oportunidades operacionais, avaliando **P**erformance, **I**nformação, **E**conomia, **C**ontrollo, **E**ficiência e **S**erviço.

**Entrevistas:** Diálogos dirigidos (estruturados ou semiestruturados) entre o analista e stakeholders para captar necessidades de alto nível, fluxos de trabalho e restrições específicas.

**Brainstorming:** Sessão aberta de ideação criativa destinada a encorajar ideias inovadoras e heterodoxas, onde a crítica imediata é totalmente proibida, focando-se na quantidade inicial para filtragem e classificação posterior.

## 8. Atributos e Fatores de Qualidade (ISO/IEC 25010)

---

A qualidade de software é medida pelo cumprimento de critérios não funcionais que asseguram que o software opere de forma fiável, segura e eficiente:

### **Interoperabilidade**

Capacidade de dois ou mais sistemas partilharem informações, serviços ou dados de maneira uniforme de acordo com metodologias e protocolos de rede padronizados.

### **Confiabilidade**

Probabilidade e garantia de um sistema funcionar e cumprir com o desempenho esperado sob condições operacionais determinadas sem registar falhas críticas.

### **Portabilidade**

Medida de esforço necessária para adaptar e migrar uma aplicação de um determinado ecossistema de software ou infraestrutura física para outro.

### **Usabilidade**

Nível de facilidade de aprendizagem, manuseamento, clareza das interfaces de utilizador e eficácia na execução de tarefas rotineiras do utilizador humano.



## 9. Boas Práticas de Design de Classes

No desenho estrutural de software com UML e POO, as decisões de design centram-se na obtenção de sistemas de fácil manutenção e expansão. Para atingir este objetivo, o design das classes deve focar no equilíbrio entre dois conceitos:

**Acoplamento:** Mede o grau de dependência entre as classes de um sistema. Quanto mais forte o acoplamento, mais difícil se torna alterar uma classe sem que tal resulte em erros nas demais que dela dependem. A boa prática prega o **Baixo Acoplamento** (desacoplar as dependências utilizando interfaces).

**Coesão:** Mede o quanto as responsabilidades e tarefas de uma classe estão integradas sob um propósito ou domínio lógico específico. O ideal é obter **Alta Coesão** (uma única responsabilidade bem delimitada).

Além disso, na representação de modelos de dados, definem-se as formas de relacionamento: a **Associação** (ligação genérica de uso entre classes), a **Agregação** (relacionamento de todo-parte fraco, onde a parte existe independentemente do todo) e a **Composição** (relacionamento de todo-parte forte, onde o ciclo de vida da parte é estritamente gerido pelo todo).

# 10. Estruturas de Algoritmos e Tipologia de Dados

## 10.1. Tipos Primitivos de Dados

| Tipo de Dado     | Descrição                                                           | Exemplos              |
|------------------|---------------------------------------------------------------------|-----------------------|
| Inteiro          | Valores numéricos sem frações decimais.                             | 0, 100, -250          |
| Real             | Valores numéricos que possuem frações ou casas de precisão decimal. | 3.1415, -45.6         |
| Literal (String) | Coleções ou sequências de caracteres alfanuméricos e símbolos.      | "Utilizador", "12345" |
| Lógico (Boolean) | Valores de álgebra binária que representam estados de validade.     | VERDADEIRO, FALSO     |

## 10.2. Regras de Atribuição e Operações

Sistemas lógicos seguem regras formais de conversão de tipos (coerção ou fundição). Por exemplo, mover um número inteiro para uma variável de tipo real é permitido implicitamente na maioria das linguagens. Contudo, tentar atribuir um valor real (com precisão fracionária) diretamente a uma variável declarada como inteira acarreta a perda da parte fracionária (truncção) ou um erro de compilação/execução de tipo inválido. Outra regra matemática elementar em computação é que a **divisão por zero é estritamente proibida**, gerando uma exceção de interrupção imediata da execução se não for prevenida logicamente.